
Mid-Air Shape Recognition in VR Using Procrustes Analysis

Author:

Dhruv Bhojaraj

July 17, 2024

Abstract

With the advent of Virtual Reality as a mainstream medium for immersive interaction with content, this project aims to explore a different method to recognize and administer feedback based on input. Methods that provide the user with control over input in VR environments include virtual keyboards, hand gestures, and eye focus interaction. This project was conceived as an exploration of how another method of input can be implemented and handled. This report is a culmination of what I created and my thoughts about the project.

Contents

1	Introduction	1
1.1	Scope	1
1.2	Objectives	1
2	Theory	2
2.1	Virtual Reality Input Device	2
2.2	The Recognition Algorithm	2
2.2.1	Procrustes Analysis	3
2.2.2	Detailed Theory and Implementation	3
2.2.3	Mathematical Formulation	4
2.2.4	Algorithmic Implementation	5
2.2.5	Application in This Project	6
3	Implementation	7
3.1	Theory	7
3.2	Application	8
3.3	The Wand Example	8
3.4	The Letter Example	9
4	Conclusion	11

1 Introduction

Virtual Reality is one of the many mediums available for users to interact with data/content. It is unique in that it provides a completely different experience with interaction and consumption of media, compared to most standard methods available today.

Being a device that allows users to experience and interact with the data/content in a 3D environment, that induces a sense of space and perspective unlike any 2D flat screen mediums, it holds a lot of different criteria and perspectives about how it should be utilized, creating an entirely different field of study in User Interaction.

Virtual Reality induces a immersive experience by allowing users to move their heads and hands to view any perspective at will, and interaction with virtual objects as a method of consuming media. This whole set of interactions is hence a subject of study, especially how said users can/will/should interact with the objects and environment around them.

Hence, this report would be me documenting what i did during the project in an informal matter, with the intent of sharing it with others.

1.1 Scope

The project goal is to explore and learn how to create a method of input that is unlike normal methods. It should not be like a virtual keyboard, but more like gestures.

1.2 Objectives

The objective of this project is to take input from the user, in terms of the space/path their hand has followed, and determine what the path is, and hence allow for an intended associated effect to be fired off.

2 Theory

In this section, several related concepts will be explained that can be the basis of the project

2.1 Virtual Reality Input Device

A Virtual reality input device, commonly known as a VR Headset, is composed of 3 main components.

- The HMD
- The Left and Right Controllers

The HMD, also known as Head Mounted Display, is the output device that contains a display, lenses, and sensors. Its primary purpose is to allow the viewing from a virtual stereoscopic camera, as well as provide sensor data like head rotation and movement to the virtual world for processing.

The Controllers are the primary inputs in a VR interaction system. It tracks the position and rotation of itself to be processed and hence visualised in the Virtual 3D space. It contains 1 joystick, 2 buttons, a Trigger Button(Index finger) and a Grip Button(Middle finger). The shape made to be ergonomic and efficient for tracking purposes. The controllers themselves show how different interactions can be in comparison to normal touchscreen devices, letting the user use their thumbs, index and middle fingers to do different things.

2.2 The Recognition Algorithm

This Project utilizes the Procrustes Analysis Algorithm to process user input. This approach was taken in order to take into consideration the direction the user is facing, any kind of human errors in terms of shape and rotation, as well as the size of the shape drawn, due to differences in preference from person to person.

2.2.1 Procrustes Analysis

Procrustes Analysis is a form of statistical shape analysis used to analyze the distribution of a set of shapes. The name "Procrustes" comes from Greek mythology, referring to a bandit who made his victims fit his bed by either stretching them or cutting off their legs.

2.2.2 Detailed Theory and Implementation

In Procrustes Analysis, shapes are aligned to each other through a series of transformations: translation, scaling, and rotation. This process minimizes the sum of squared distances between corresponding points, effectively "superimposing" one shape onto another. Let's delve into each step in detail:

1. Translation

Objective: Move the shapes to a common origin, typically by centering them at (0,0).

Process:

1. Calculate the centroid of each shape by averaging all landmark coordinates:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$$

where (x_i, y_i) are the coordinates of the i -th landmark and n is the total number of landmarks.

2. Subtract the centroid coordinates from each landmark:

$$(x'_i, y'_i) = (x_i - \bar{x}, y_i - \bar{y})$$

2. Scaling

Objective: Resize the shapes to a common scale, often normalizing them to unit size.

Process:

1. Calculate the centroid size S for each shape:

$$S = \sqrt{\sum_{i=1}^n ((x'_i)^2 + (y'_i)^2)}$$

2. Divide each coordinate by the centroid size:

$$(x''_i, y''_i) = \left(\frac{x'_i}{S}, \frac{y'_i}{S}\right)$$

3. Rotation

Objective: Rotate one shape to align with the other, minimizing the sum of squared distances between corresponding points.

Process:

1. Let X and Y be the matrices of coordinates for the two shapes after translation and scaling.
2. Compute the covariance matrix: $A = X^T Y$
3. Perform Singular Value Decomposition (SVD) on A : $A = U \Sigma V^T$
4. Compute the rotation matrix: $R = V U^T$
5. Apply the rotation to X : $X_{rotated} = X R$

2.2.3 Mathematical Formulation

Let X and Y be two shapes represented by matrices where each row corresponds to a landmark point. The Procrustes distance between X and Y is defined as:

$$d_P(X, Y) = \inf_{\beta, R, t} \|Y - \beta X R - 1_n t^T\|_F$$

where:

- $\beta > 0$ is a scaling factor

- R is a rotation matrix
- t is a translation vector
- 1_n is a vector of ones
- $\|\cdot\|_F$ denotes the Frobenius norm

2.2.4 Algorithmic Implementation

The Procrustes Analysis algorithm can be summarized as follows:

1. **Input:** Two shapes X and Y represented as matrices of landmark coordinates
2. **CenterShapes:**
 - Calculate centroids of X and Y
 - Subtract centroids from respective shapes to center them
3. **ScaleShapes:**
 - Calculate centroid sizes of centered shapes
 - Divide coordinates by centroid sizes to normalize
4. **ComputeRotationMatrix:**
 - Compute covariance matrix $A = X^T Y$
 - Perform SVD: $A = U \Sigma V^T$
 - Calculate rotation matrix $R = V U^T$
5. **AlignShapes:** Apply rotation to X : $X_{aligned} = X R$
6. **Output:** $X_{aligned}$ and Y (now optimally aligned)

This procedure minimizes the Procrustes distance between X and Y , effectively aligning the shapes for comparison or further analysis.

2.2.5 Application in This Project

In our project, Procrustes Analysis allows us to:

- Normalize user input regardless of the size of the drawn shape
- Compensate for rotational differences due to user orientation
- Account for minor shape distortions caused by human error in drawing

Implementation Details:

1. User Input Preprocessing:
 - Convert drawn shape into a set of landmark points
 - Apply noise reduction techniques to smooth out irregularities
2. Template Matching:
 - Maintain a database of pre-defined shape templates
 - For each user input, perform Procrustes Analysis with all templates
 - Select the template with the smallest Procrustes distance as the match
3. Recognition Refinement:
 - Implement a threshold for acceptable Procrustes distances
 - Create the templates based on the average points at each index in the sample set

This approach significantly improves the robustness of our recognition algorithm, making it more adaptable to various user inputs and environmental conditions. By accounting for variations in size, rotation, and minor distortions, we can achieve a high degree of accuracy in shape recognition across a diverse user base.

3 Implementation

The Algorithm was implemented in the Unity Real Time Rendering Engine, for its good compatibility for VR Development.

3.1 Theory

- When the Grip Button is held, create a List of 3D Coordinates and add the position of the hand at that frame
- At each frame while the button is held, check if the current position of the hand is at a distance above a certain specified threshold away from the previous registered point, if so, store the position, otherwise pass.
- Once the Grip button is let go, stop checking/collecting the hand positions.
- Using Unity's Spline Package, generate a spline with all the points in the list, with auto smooth on.
- Execute the Laplacian smoothing algorithm on the spline to smooth out any unexpected bumps in the line
- Convert the Spline into an array of Vector3, using a fixed resolution. (60 used in this project).
 - The reason behind using a spline and converting it to an array later is because of the threshold. If the user draws a small shape, we can expect about 10 points, while if the user draws a big shape, it can be 75 points.
 - In order to normalize them, we convert the points into a spline, with auto smooth, and then equally sample the spline at a specific resolution, creating an array of positions that are fixed and can be compared relatively.
- Perform the Procrustes analysis to compare the sampled array to the database of shapes available, and deem the lowest distance difference as a match

Regarding the Dataset used to compare the shapes. For each shape that needs to be checked, the shape is drawn multiple times(15-25), converted into a 60 value array as mentioned earlier, and then the average for each index is calculated and stored to create the ideal shape.

3 Implementation

This method allows for comparison with the "perfect humanly drawn shape", with the averaging removing any minor defects, but still allowing it to resemble something human drawn.

3.2 Application

3.3 The Wand Example

Upon creating the project, i thought that a input mechanic like this can contribute to creating something like spell casting from the Harry Potter series. Hence the 1st example being a showcase of a spell system.

The Dataset

Only 3 shapes were recorded and stored for comparison.

- Square : Spell that destroys an object
- Triangle : Spell that levitates and spins an object
- Circle : Spell that teleports the player

The Environment

I picked up a CC0 environment from online, to showcase the mechanic, with the 3D modelled wand made by myself.



Figure 3.1: Starting the Application



Figure 3.2: Drawing a Circle to Teleport



Figure 3.3: Drawing a Triangle

3.4 The Letter Example

After the above example, i began to theorise what practical uses could an input mechanic like this in real world applications, coming to the conclusion that i can try performing letter recognition as an alternative to a virtual keyboard.

The Dataset

All the 26 alphabet were stored this time, with a few special conditions, all of which are in lower case.

Due to the nature of the algorithm, i needed all letter to be represented in 1 line, which the English scripting language does not comply with. To combat this issue, i opted for using cursive English, to mitigate most of the problems that may arise. That aside, here are some cases that are interesting to point out or require an exception of sorts.

- J : chose to ignore the dot in j
- I : decided to use uppercase I since a lowercase i in cursive was too similar to l
- W : since rotations are cancelled out, m and w may be mistaken for each other. to solve this, w is given a curve at the end (like in cursive)
- Y : to ensure j and y are not mistaken for each other, y has its ending taken to the top
- N : has a small curve at the end to be differentiated from u
- P: was drawn to look more like ρ so that it's not mistaken with d.
- S : cursive s looks like an 8.
- X : is represented with x .

The Environment

I designed the environment to resemble Meta's own shutdown environment. The images display the match percentage for each letter, along with the word builder on the left, as a demo use case of the system.

This project can be found here

3 Implementation



Figure 3.4: Adding Letter “D”



Figure 3.5: Adding Letter “V”

4 Conclusion

Successfully Implemented the Procrustes analysis algorithm to allow for drawing shapes in the air as a method of input.

This project is considered to be a success in my eyes, despite the drawbacks that are natural with the algorithm.

However, with this method of implementation, it would be very easy to add new shapes to the comparison list, and i am curious about other possible applications of this mechanic. Overall, this project was quite fun and i learnt a lot out of it, eventually, i would want to learn proper gesture detection, and then revisit this project.